MEMORIA PRÁCTICA 1

GESTIÓN DE PROCESOS Y ARCHIVOS. COMUNICACIÓN ENTRE PROCESOS: TUBERÍAS Y MEMORIA COMPARTIDA

Autor: *Alejandro Andreu Castillo*

Universidad: *Universidad de Alicante*

Facultad: *Escuela Politécnica Superior*

Asignatura: *Sistemas Operativos*

Profesor: *Alejandro Serrano Cases*

Grupo: *401*

Fecha: *11/10/26*

0. ÍNDICE GENERAL

1.1. Ejercicio a) ***3***

1.1.1. Enunciado del ejercicio *3*

1.1.1.1. Estructura del Árbol de Procesos *3*

1.1.1.2. Resultado Esperado *3*

1.1.2 Identificación y Justificación de Tareas *4*

1.1.3. Diagrama de Secuencia *5*

1.1.4. Análisis Detallado del Código Fuente *7*

1.2. Ejercicio b) ***11***

1.2.1. Enunciado del ejercicio *11*

1.2.1.1. Estructura del Árbol de Procesos *11*

1.2.1.2. Resultado Esperado *12*

1.2.2. Identificación y Justificación de Tareas *12*

1.2.3. Diagrama de Secuencia *14*

1.2.4. Análisis Detallado del Código Fuente *16*

2. Ejercicio 2 ..... ***21***

2.1. Enunciado del Ejercicio *21*

2.1.1. Estructura del Árbol de Procesos *21*

2.1.2. Resultado Esperado *21*

2.2. Identificación y Justificación de Tareas *22*

2.3. Diagrama de Secuencia *23*

2.4. Análisis Detallado del Código Fuente *25*

3. Ejercicio 3 ..... ***29***

3.1. Enunciado del Ejercicio *29*

3.1.1. Estructura del Árbol de Procesos *29*

3.1.2. Resultado Esperado *29*

3.2. Identificación y Justificación de Tareas *30*

3.3. Diagrama de Secuencia *31*

3.4. Análisis Detallado del Código Fuente *33*

4. Anexo ***37***

4.1 Código fuente *37*

4.2 Herramientas Utilizadas *37*

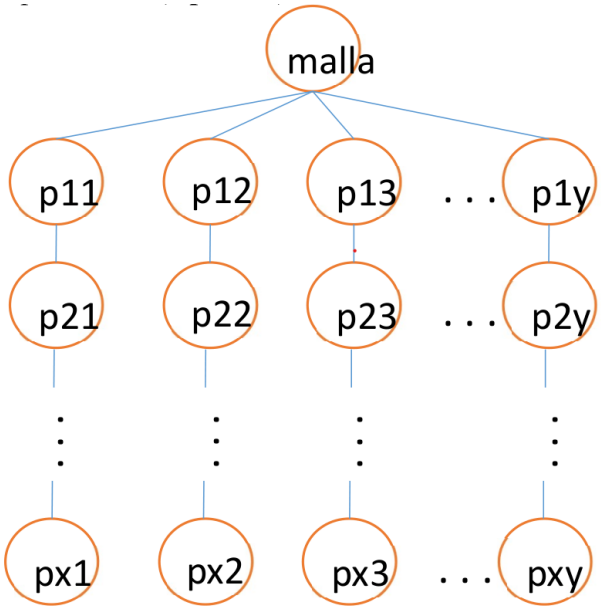
4.3 Uso de herramientas IA *37*

1.1. Ejercicio a)

1.1.1. Enunciado del ejercicio

Realiza un programa llamado **malla.c** que produzca el siguiente árbol de procesos. El programa recibirá dos argumentos 'x' e 'y' que representan el número de filas y columnas. Para comprobar la estructura de procesos que realmente estamos creando debemos emplear la llamada al sistema "*pstree –c*".

1.1.1.1. Estructura del Árbol de Procesos



1.1.1.2. Resultado Esperado

El programa debe mostrar por la salida estándar la siguiente información:

```
$ malla 3 4
malla─┬─malla───malla───malla
      ├─malla───malla───malla
      ├─malla───malla───malla
      ├─malla───malla───malla
      └─pstree
```

1.1.2 Identificación y Justificación de Tareas

La arquitectura del programa se deriva de los requisitos del enunciado en estas tareas clave.

1.1.2.1. Recepción y Validación de Argumentos

El programa requiere obtener los parámetros que introduzca el usuario por terminal. La función ***leerArgumentos()*** se encarga de verificar que se hayan introducido correctamente, garantizando el correcto funcionamiento del programa.

1.1.2.2. Generación de la Jerarquía de Procesos

La creación del árbol se divide en dos fases:

- Primero, en la creación de las columnas con ***creaHorizontal()***.
- Segundo, en la creación de las filas con ***creaVertical()***.

Cada proceso debe crear a su hijo y, tras ello, bloquearse.

1.1.2.3. Temporización sin Bloqueo

Se implementa un mecanismo de espera asíncrona. El último proceso hace una alarma con ***alarm()*** y suspende su ejecución con ***pause()***.

1.1.2.4. Ejecución de la Orden

Al expirar el tiempo de la alarma, el último proceso avisa al super padre (proceso raíz). Dicho padre ejecuta el "*pstree -c*" desde un proceso hijo externo mediante ***fork()*** y ***execlp()***. Esto se hace porque exec sustituye la imagen del proceso y, por tanto, se debe crear un hijo que lo ejecute para que el proceso principal no desaparezca.

1.1.2.5. Destrucción Controlada del Árbol

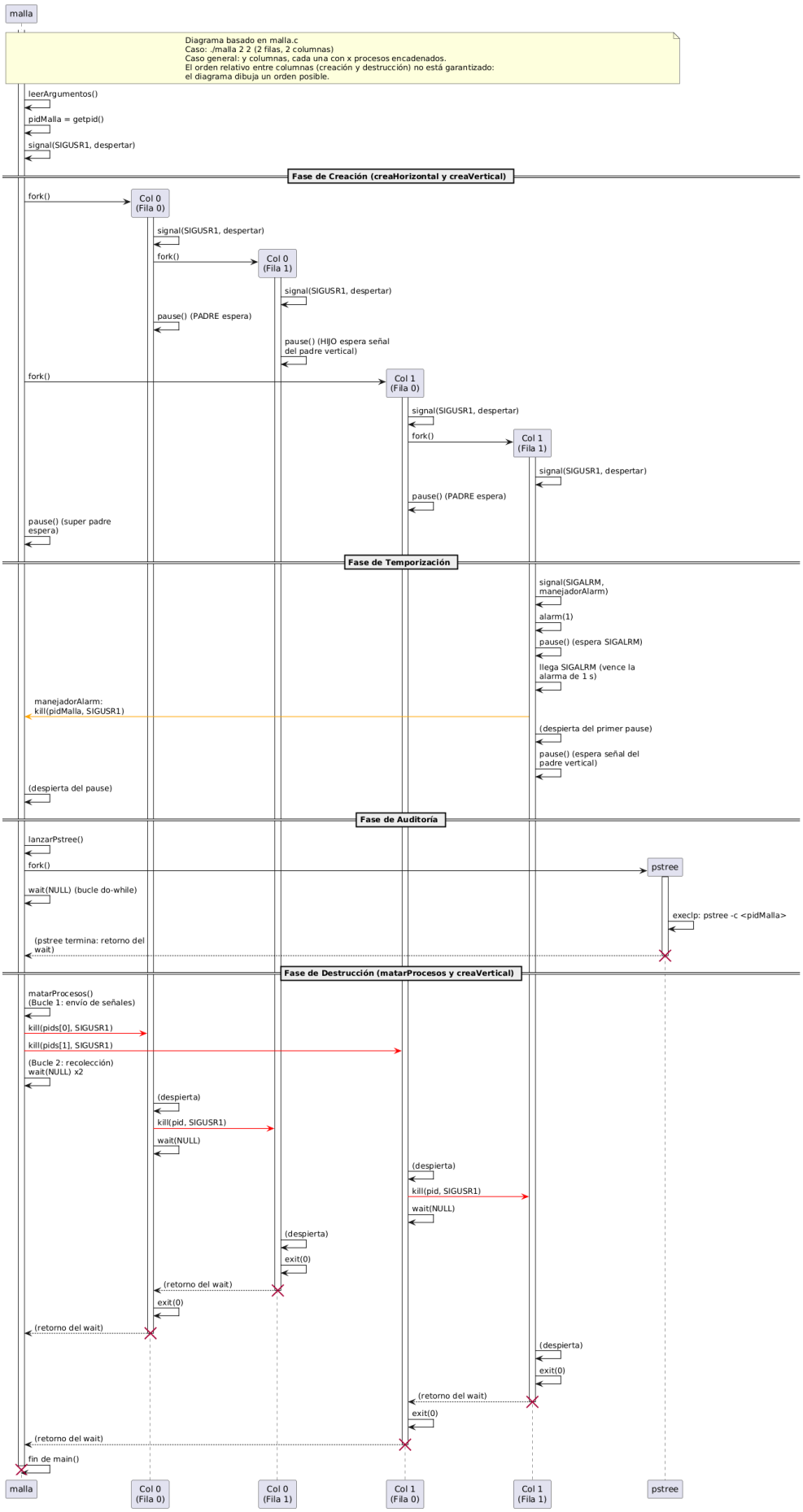
Para cumplir con que el árbol se destruya de abajo a arriba, se utiliza una cascada de señales. Tras el aviso del último proceso al super padre, este notifica a la primera fila, y el aviso desciende por la cadena. Cada padre reenvía la señal **SIGUSR1** a su hijo directo y ejecuta ***wait(NULL)***, bloqueándose y esperando a la muerte del descendiente para finalizar.

1.1.3. Diagrama de Secuencia

El siguiente diagrama muestra el ciclo de vida completo de malla para el caso *./malla 2 2* (2 filas y 2 columnas).

La malla crea las columnas con fork y cada una crea a su hijo, y todos quedan en **pause** esperando señal. El último proceso de la última columna programa una **alarm(1)** y, al sonar, su manejador avisa a la malla con **SIGUSR1** (flecha naranja).

La malla lanza entonces "*pstree -c*" y, cuando termina, inicia la destrucción: envía **SIGUSR1** a la cabeza de cada columna y cada padre se lo pasa a su hijo y espera con wait antes de morir (flechas rojas). Así ningún padre muere antes que sus hijos y la malla es la última en terminar.


malla
Diagrama basado en malla.c
Caso: ./malla 2 2 (2 filas, 2 columnas)
Caso general: y columnas, cada una con x procesos encadenados.
El orden relativo entre columnas (creación y destrucción) no está garantizado:
el diagrama dibuja un orden posible.
leerArgumentos()
pidMalla = getpid()
signal(SIGUSR1, despertar)
Fase de Creación (creaHorizontal y creaVertical)
fork()
Col 0 (Fila 0)
signal(SIGUSR1, despertar)
fork()
Col 0 (Fila 1)
signal(SIGUSR1, despertar)
pause() (PADRE espera)
pause() (HIJO espera señal del padre vertical)
fork()
Col 1 (Fila 0)
signal(SIGUSR1, despertar)
fork()
Col 1 (Fila 1)
signal(SIGUSR1, despertar)
pause() (PADRE espera)
pause() (super padre espera)
Fase de Temporización
signal(SIGALRM, manejadorAlarm)
alarm(1)
pause() (espera SIGALRM)
llega SIGALRM (vence la alarma de 1 s)
manejadorAlarm: kill(pidMalla, SIGUSR1)
(despierta del primer pause)
pause() (espera señal del padre vertical)
(despierta del pause)
Fase de Auditoría
lanzarPstree()
fork()
pstree
wait(NULL) (bucle do-while)
execlp: pstree -c <pidMalla>
(pstree termina: retorno del wait)
Fase de Destrucción (matarProcesos y creaVertical)
matarProcesos()
(Bucle 1: envío de señales)
kill(pids[0], SIGUSR1)
kill(pids[1], SIGUSR1)
(Bucle 2: recolección)
wait(NULL) x2
(despierta)
kill(pid, SIGUSR1)
wait(NULL)
(despierta)
kill(pid, SIGUSR1)
wait(NULL)
(despierta)
exit(0)
(retorno del wait)
exit(0)
(retorno del wait)
(despierta)
exit(0)
(retorno del wait)
exit(0)
(retorno del wait)
fin de main()
malla
Col 0 (Fila 0)
Col 0 (Fila 1)
Col 1 (Fila 0)
Col 1 (Fila 1)
pstree

1.1.4. Análisis Detallado del Código Fuente

1.1.4.1. Funciones Globales y de Utilidad

- Función *leerArgumentos()*: su propósito es recibir e interpretar los argumentos recibidos del usuario por la línea de comandos. Valida que la cuadrícula introducida tenga sentido y sea posible, y almacena los valores en los punteros pasados como argumento.

- Llamadas al sistema utilizadas: *exit()*.

```
void leerArgumentos(int nArgs, char *args[], int *x, int *y) {
    if (nArgs != 3 || atoi(args[1]) < 1 || atoi(args[2]) < 1) {
        printf("Error. Debes introducir un número válido de argumentos
que sean > 0 \n");
        exit(1);
    }
    else {
        *x = atoi(args[1]); // FILAS
        *y = atoi(args[2]); // COLUMNAS
    }
}
```

- Funciones *despertar()* y *manejadorAlarm()*: sirven para interrumpir los ***pause()***. *despertar()* está vacío porque simplemente sirve para hacer que la señal no mate al proceso. *manejadorAlarm()* envía **SIGUSR1** al proceso cuyo PID guarda la variable global pidMalla.

- Llamadas al sistema utilizadas: *kill()*.

1.1.4.2. Procesos nodo (hijos)

- Función *creaHorizontal()*: crea la primera fila de la malla. Para cada columna, ejecuta ***creaVertical()*** de forma iterativa, mientras que el padre almacena sus identificadores (PIDs)

- Llamadas al sistema utilizadas: *fork(), perror(), exit()*.

```
void creaHorizontal(int ncolumnas, int nfilas, pid_t pids[]) {
    for (int i=0; i<ncolumnas; i++) {
        pid_t pid = fork();
        if (pid == 0) {
            int ultima = 0;
            if (i == ncolumnas-1) {
```

```
                ultima = 1;
            }
            creaVertical(nfilas,ultima);
        }
        else if (pid == -1) {
            perror("Error en fork");
            exit(1);
        }
        pids[i] = pid;
    }
}
```

- **Función *creaVertical()*:** implementa un modelo iterativo para que cada padre cree a un hijo y se suspenda con ***pause()***. Al despertar, notifica a su descendiente directo y se bloquea con ***wait(NULL)***, garantizando que el hijo muera antes que él. Si el proceso resulta ser el último de la malla, programa el temporizador de 1 segundo antes de suspenderse.

- Llamadas al sistema utilizadas: *signal(), fork(), pause(), kill(), wait(), exit(), alarm(), perror()*.

```
void creaVertical(int nfilas, int esUltimaColumna) {
    signal(SIGUSR1, despertar);
    for (int i=1; i<nfilas; i++) {
        pid_t pid = fork();
        if (pid > 0) {
            pause();
            kill(pid, SIGUSR1);
            wait(NULL);
            exit(0);
        }
        else if (pid == -1) {
            perror("Error en fork");
            exit(1);
        }
        signal(SIGUSR1, despertar);
    }
    if (esUltimaColumna) {
        signal(SIGALRM, manejadorAlarm);
        alarm(1);
        pause();
    }
    pause();
    exit(0);
}
```

1.1.4.3. Proceso super padre (raíz)

- Función *lanzarPstree()*: duplica el proceso padre y ejecuta el "*pstree -c*". Utiliza un bucle do-while para asegurar que el ***wait(NULL)*** recoja al proceso correcto (sirve para el correcto funcionamiento cuando solo se introduce una fila para n columnas).

- Llamadas al sistema utilizadas: *fork(), execlp(), perror(), exit(), wait()*.

```
void lanzarPstree() {
    pid_t pid = fork();
    if (pid == -1) {
        perror("Error en fork");
        exit(1);
    }
    else if (pid == 0) {
        char buffer[20];
        sprintf(buffer, "%d", pidMalla);
        execlp("pstree","pstree","-c",buffer,NULL);
        perror("Ejecución pstree");
        exit(1);
    }
    pid_t fin;
    do {
        fin = wait(NULL);
    } while (fin != pid && fin != -1);
}
```

- Función *matarProcesos()*: envía la orden de muerte a la primera fila de procesos y después sincroniza con wait(NULL). Esto se hace mediante dos estructuras iterativas que finalizan con el número de columnas indicadas como parámetro.

- Llamadas al sistema utilizadas: *kill(), wait()*.

1.1.4.4. Función Principal

- <u>Función *main()*</u>: punto de entrada que dirige y organiza todo el código. Registra su manejador, crea el array de PIDs para las columnas y llama a ***creaHorizontal()*** para generar el árbol de procesos. Posteriormente, se bloquea con ***pause()*** a la espera de la alarma, invoca ***lanzarPstree()*** y, finalmente, ejecuta ***matarProcesos()***.

- Llamadas al sistema utilizadas: *getpid(), signal(), pause()*.

```
int main(int argc, char *argv[]) {
    int nfilas, ncolumnas;
    leerArgumentos(argc, argv, &nfilas, &ncolumnas);

    pidMalla = getpid();

    signal(SIGUSR1, despertar);

    pid_t pids[ncolumnas];
    creaHorizontal(ncolumnas, nfilas, pids);

    pause();
    lanzarPstree();

    matarProcesos(ncolumnas, pids);
}
```

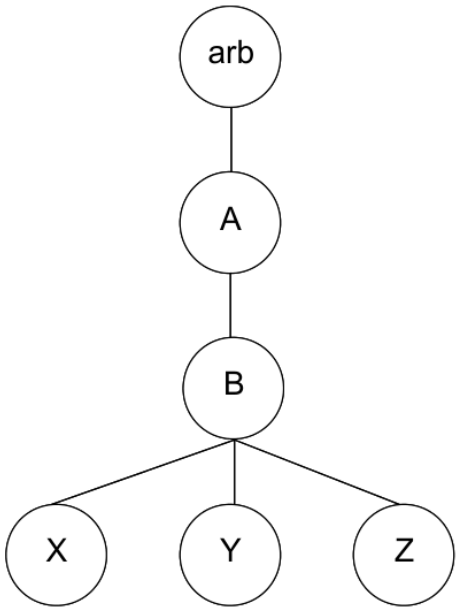
1.2. Ejercicio b)

1.2.1. Enunciado del ejercicio

Realiza un programa llamado **ejec.c** que reciba un argumento. El programa tendrá que generar el árbol de procesos que se indica y llevar a cabo la funcionalidad que se describe a continuación. El proceso Z, transcurridos los segundos indicados por el argumento, enviará una señal al proceso A. El proceso A, al recibir la señal, ejecutará el comando "pstree". El proceso Z no puede utilizar el comando "sleep", por lo que el proceso Z debe planificarse una alarma con los segundos indicados por el argumento.

Se deberá controlar la correcta destrucción del árbol (los padres no pueden morir antes que los hijos).

1.2.1.1. Estructura del Árbol de Procesos



1.2.1.2. Resultado Esperado

El programa debe mostrar por la salida estándar la siguiente información:

```
$ ejec 15
Soy el proceso ejec: mi pid es 751
Soy el proceso A: mi pid es 752. Mi padre es 751
Soy el proceso B: mi pid es 753. Mi padre es 752. Mi abuelo es 751
Soy el proceso X: mi pid es 754. Mi padre es 753. Mi abuelo es 752. Mi bisabuelo es 751
Soy el proceso Y: mi pid es 755. Mi padre es 753. Mi abuelo es 752. Mi bisabuelo es 751
Soy el proceso Z: mi pid es 756. Mi padre es 753. Mi abuelo es 752. Mi bisabuelo es 751
/*Tras un intervalo de 15 segundos aparecerá*/
/*Resultado del comando "pstree" */
Soy Z (756) y muero
Soy Y (755) y muero
Soy X (754) y muero
Soy B (753) y muero
Soy A (752) y muero
Soy ejec (751) y muero
```

1.2.2. Identificación y Justificación de Tareas

La resolución del ejercicio se ha estructurado de forma lógica y con las partes bien diferenciadas y altamente encapsuladas. Las tareas a realizar son las siguientes.

1.2.2.1. Validación de parámetros

Para controlar la recepción adecuada de los parámetros se ha implementado una función de lectura **leerArgumento()**, encargada de procesar la entrada por la línea de comandos. Con ello, se verifica que se proporcione un solo argumento y de tipo entero (mayor que 0) que se corresponde con el tiempo de duración de la alarma del proceso Z.

1.2.2.2. Construcción Jerárquica del Árbol

El árbol tiene tanto una estructura vertical (ejec → A → B) como una estructura horizontal (B → X, Y, Z) desde el último proceso. Se controla que el hijo lo esté generando solo el padre correspondiente en la estructura vertical; y que el mismo padre cree a los tres en la estructura horizontal.

1.2.2.3. Temporización Asíncrona

Cumpliendo la restricción del enunciado de que el proceso Z no puede utilizar la función ***sleep()***, se planifica una alarma mediante ***alarm(tiempo)*** junto a una suspensión pasiva con ***pause()***. Esto permite mantener en espera el proceso hasta a recepción de la señal **SIGALRM**.

1.2.2.4. Comportamiento Reactivo y Estructura

A la recepción de la señal de alarma, el proceso Z notifica al proceso A con **SIGUSR1**. Al recibirla, A llama a la función ***lanzarPstree()***, la cual crea un hijo externo para que este ejecute el comando "*pstree -c*", invocando a ***execlp()***. Como ya explicado en el ejercicio anterior, se hace porque "exec" sustituye la imagen del proceso y, por tanto, se debe crear un hijo para que el proceso principal no desaparezca.

1.2.2.5. Cascada y Destrucción Controlada

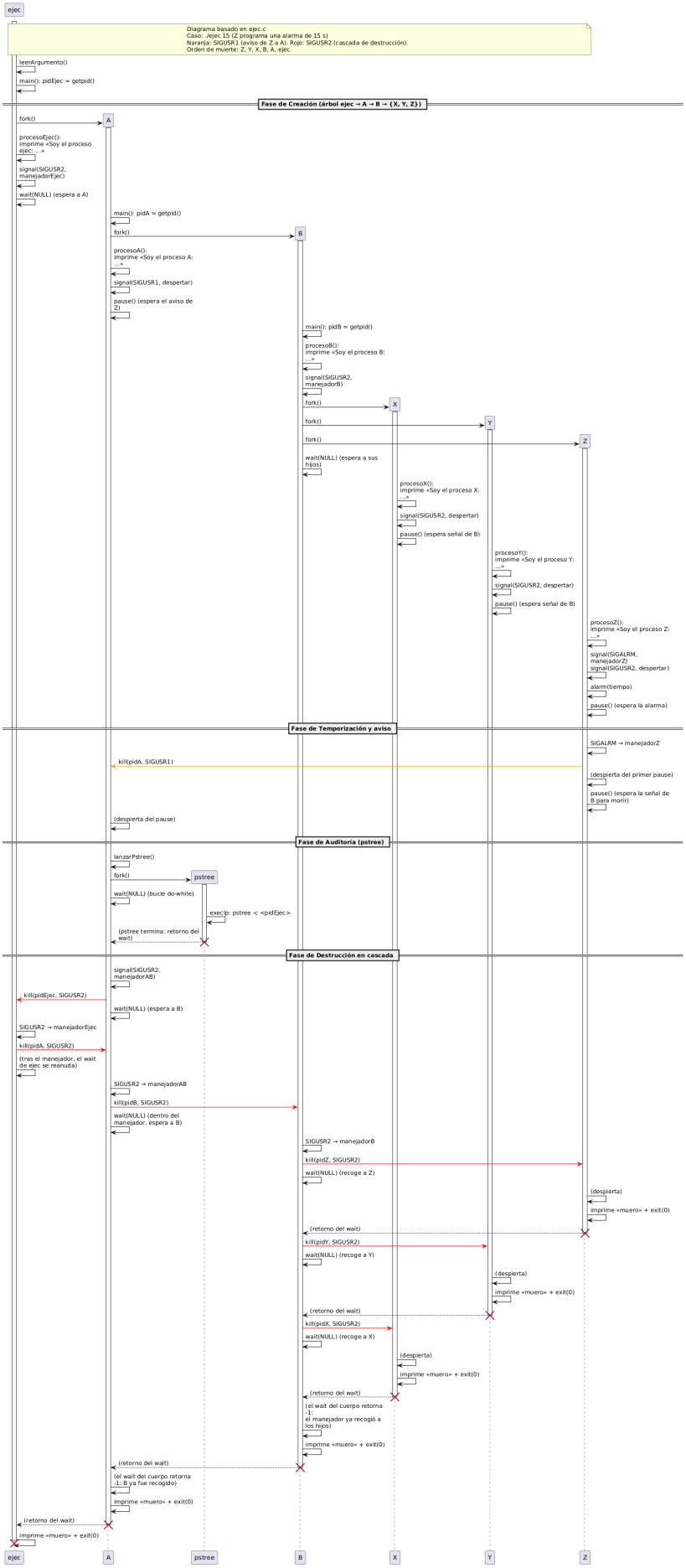
Para evitar procesos huérfanos (hijos que queden vivos y fuera de control) y cumplir con la condición fundamental, se establece un flujo de señales en sentido descendente (ejec → A → B). Esta cascada se coordina a través de los manejadores. El proceso B, mediante un manejador, envía la señal de fin ***kill()*** a sus hijos (X, Y, Z) y se sincroniza mediante llamadas ***wait(NULL)***.

1.2.3. Diagrama de Secuencia

El siguiente diagrama muestra el ciclo de vida completo de ejec para el caso *./ejec 15*.

El programa crea el árbol *ejec → A → B → {X, Y, Z}* con **fork** y cada proceso registra sus manejadores de señal y se queda en **pause**. Z no puede usar sleep, así que programa una **alarm** con los segundos recibidos como argumento. Al sonar, su manejador (*manejadorZ*) envía **SIGUSR1** a A (flecha naranja). A lanza entonces "*pstree -c*" y, cuando termina, inicia la destrucción: avisa a ejec con **SIGUSR2**, que se lo devuelve a A, y A se lo pasa a B. B recoge a sus hijos uno a uno, en el orden Z, Y, X, enviando **SIGUSR2** y esperando con **wait** a cada uno (flechas rojas).

Cada padre solo muere cuando ha recogido a su hijo, de modo que ningún padre muere antes que sus hijos y ejec es el último en terminar.


ejec
Diagrama basado en ejec.c
Caso: ./ejec 15 (Z programa una alarma de 15 s)
Naranja: SIGUSR1 (aviso de Z a A). Rojo: SIGUSR2 (cascada de destrucción).
Orden de muerte: Z, Y, X, B, A, ejec.
leerArgumento()
main(): pidEjec = getpid()
Fase de Creación (árbol ejec → A → B → {X, Y, Z})
fork()
A
procesoEjec(): imprime «Soy el proceso ejec. ...»
signal(SIGUSR2, manejadorEjec)
wait(NULL) (espera a A)
main(): pidA = getpid()
fork()
B
procesoA(): imprime «Soy el proceso A: ...»
signal(SIGUSR1, despertar)
pause() (espera el aviso de Z)
main(): pidB = getpid()
procesoB(): imprime «Soy el proceso B: ...»
signal(SIGUSR2, manejadorB)
fork()
X
fork()
Y
fork()
Z
wait(NULL) (espera a sus hijos)
procesoX(): imprime «Soy el proceso X: ...»
signal(SIGUSR2, despertar)
pause() (espera señal de B)
procesoY(): imprime «Soy el proceso Y: ...»
signal(SIGUSR2, despertar)
pause() (espera señal de B)
procesoZ(): imprime «Soy el proceso Z: ...»
signal(SIGALRM, manejadorZ)
signal(SIGUSR2, despertar)
alarm(tiempo)
pause() (espera la alarma)
Fase de Temporización y aviso
SIGALRM → manejadorZ
kill(pidA, SIGUSR1)
(despierta del primer pause)
pause() (espera la señal de B para morir)
(despierta del pause)
Fase de Auditoría (pstree)
lanzarPstree()
fork()
pstree
wait(NULL) (bucle do-while)
execlp: pstree < <pidEjec>
(pstree termina: retorno del wait)
Fase de Destrucción en cascada
signal(SIGUSR2, manejadorAB)
kill(pidEjec, SIGUSR2)
wait(NULL) (espera a B)
SIGUSR2 → manejadorEjec
kill(pidA, SIGUSR2)
(tras el manejador, el wait de ejec se reanuda)
SIGUSR2 → manejadorAB
kill(pidB, SIGUSR2)
wait(NULL) (dentro del manejador, espera a B)
SIGUSR2 → manejadorB
kill(pidZ, SIGUSR2)
wait(NULL) (recoge a Z)
(despierta)
imprime «muero» + exit(0)
(retorno del wait)
kill(pidY, SIGUSR2)
wait(NULL) (recoge a Y)
(despierta)
imprime «muero» + exit(0)
(retorno del wait)
kill(pidX, SIGUSR2)
wait(NULL) (recoge a X)
(despierta)
imprime «muero» + exit(0)
(retorno del wait)
(el wait del cuerpo retorna -1: el manejador ya recogió a los hijos)
imprime «muero» + exit(0)
(retorno del wait)
(el wait del cuerpo retorna -1: B ya fue recogido)
imprime «muero» + exit(0)
(retorno del wait)
imprime «muero» + exit(0)
ejec
A
pstree
B
X
Y
Z

1.2.4. Análisis Detallado del Código Fuente

1.2.4.1. Funciones Globales

- Función *leerArgumento()*: es la puerta de entrada para la recepción y validación de los parámetros externos. Además, convierte la cadena de tiempo a formato numérico para su uso.

- Llamadas al sistema utilizadas: *exit()*.

- Función *comprobarFork()*: sirve como control de errores en la creación de procesos. Evalúa el PID resultante e interrumpe el programa ante un proceso creado de forma errónea.

- Llamadas al sistema utilizadas: *perror(), exit()*.

```
void comprobarFork(pid_t pid, const char *proceso) {
    if (pid == -1) {
        perror(proceso);
        exit(1);
    }
}
```

- Función *despertar()*: manejador vacío, se enlaza a señales con el único fin de interrumpir los estados de espera, pause(), de los procesos.

- Función *lanzarPstree()*: duplica el proceso padre y ejecuta el "*pstree -c*". Utiliza un bucle do-while para asegurar que el wait(NULL) recoja al proceso correcto.

- Llamadas al sistema utilizadas: *fork(), execlp(), perror(), exit(), wait()*.

1.2.4.2. Manejadores de Señales

- Función *manejadorZ()*: ejerce de disparador inicial. Captura el fin de la alarma del proceso Z y lo notifica al proceso A, enviando **SIGUSR1** al PID correspondiente (pidA).

- Llamadas al sistema utilizadas: *kill()*.

- **Función *manejadorEjec()* y *manejadorAB()*:** actúan como intermediarios en la cascada. Cuando ejec es notificado por A (al acabar el "*pstree -c*"), el *manejadorEjec()* propaga la muerte hacia A con la señal **SIGUSR2**. A su vez, el *manejadorAB*() reenvía la señal hacia B y espera con ***wait(NULL)*** a que termine.

    - Llamadas al sistema utilizadas: *kill(), wait()*.

- **Función *manejadorB()*:** para asegurar la correcta destrucción del árbol, invoca llamadas emparejadas de ***kill()*** y ***wait(NULL)*** individualmente a Z, Y y X, como se ve a continuación.

    - Llamadas al sistema utilizadas: *kill(), wait()*.

```
void manejadorB(int sig) {
    kill(pidZ, SIGUSR2);
    wait(NULL);

    kill(pidY, SIGUSR2);
    wait(NULL);

    kill(pidX, SIGUSR2);
    wait(NULL);
}
```

1.2.4.3. Procesos del Árbol

- **Procesos principales:** el ***procesoA()*** asume la responsabilidad reactiva ya que espera al aviso de Z para ejecutar ***lanzarPstree()*** y avisa de ello al super padre "ejec". Además, registra el manejador de destrucción *manejadorAB()*.

    - Llamadas al sistema utilizadas: *signal(), pause(), kill()*.

```
void procesoA() {
    printf("Soy el proceso A: mi pid es %d. Mi padre es %d\n", pidA,
pidEjec);
    signal(SIGUSR1, despertar);
    pause();
    lanzarPstree();
    signal(SIGUSR2, manejadorAB);
    kill(pidEjec, SIGUSR2);
    wait(NULL);
    printf("Soy A(%d) y muero\n", getpid());
    exit(0);
}
```

Por otro lado, el ***procesoEjec()*** permanece a la espera de la señal de A para iniciar la cascada descendente de muerte.

- Llamadas al sistema utilizadas: *signal(), wait(), exit()*.

```
void procesoEjec() {
    printf("Soy el proceso ejec: mi pid es %d\n", pidEjec);
    signal(SIGUSR2, manejadorEjec);
    wait(NULL);
    printf("Soy ejec(%d) y muero\n", getpid());
    exit(0);
}
```

- <u>Procesos intermedios</u>: el ***procesoB()*** crea la estructura horizontal de procesos y registra el manejador de destrucción de estos, *manejadorB()*. Tras ello, se suspende hasta la muerte de esos 3 procesos.

- Llamadas al sistema utilizadas: *signal(), fork(), wait(), getpid(), exit()*.

```
void procesoB(int tiempo) {
    printf("Soy el proceso B: mi pid es %d. Mi padre es %d. Mi abuelo es
%d\n", pidB, pidA, pidEjec);

    signal(SIGUSR2, manejadorB);

    pidX = fork();
    comprobarFork(pidX, "Error en fork de X");
    if (pidX == 0) {
        procesoX();
    }
    pidY = fork();
    comprobarFork(pidY, "Error en fork de Y");
    if (pidY == 0) {
        procesoY();
    }
    pidZ = fork();
    comprobarFork(pidZ, "Error en fork de Z");
    if (pidZ == 0) {
        procesoZ(tiempo);
    }

    wait(NULL);
    printf("Soy B(%d) y muero\n", getpid());
    exit(0);
}
```

- <u>Procesos hoja</u>: los ***procesoX()*** y ***procesoY()*** registran su manejador para despertar y esperan en ***pause()***.

Aclaración: ambos tienen la misma estructura: imprimen su mensaje, registran ***despertar()*** para **SIGUSR2**, esperan en ***pause()*** y mueren al recibir la señal de B

- Llamadas al sistema utilizadas: *signal(), pause(), getpid(), exit()*.

```
void procesoX() {
    printf("Soy el proceso X: mi pid es %d. Mi padre es %d. Mi abuelo es
%d. Mi bisabuelo es %d\n", getpid(), pidB, pidA, pidEjec);
    signal(SIGUSR2, despertar);
    pause();

    printf("Soy X(%d) y muero\n", getpid());
    exit(0);
}
```

Por el otro lado, el ***procesoZ()*** envía una señal al proceso A mediante el *manejadorZ()*, también registra un segundo manejador para despertar. Además, incorpora la llamada de alarma dado el tiempo establecido y requiere de dos ***pause()***: el primero para ceder la CPU durante el temporizador y el segundo para esperar la señal de la cascada.

```
void procesoZ(int tiempo) {
    printf("Soy el proceso Z: mi pid es %d. Mi padre es %d. Mi abuelo es
%d. Mi bisabuelo es %d\n", getpid(), pidB, pidA, pidEjec);

    signal(SIGALRM, manejadorZ);
    signal(SIGUSR2, despertar);

    alarm(tiempo);
    pause();
    pause();

    printf("\nSoy Z(%d) y muero\n", getpid());
    exit(0);
}
```

1.2.4.4. Función Principal

- <u>Función *main()*</u>: el bloque principal del programa se ha simplificado para que únicamente se encargue de coordinar el flujo requerido. Invoca a ***leerArgumento()***, encadena los ***fork()*** para construir el árbol de forma adecuada y delega el ciclo de vida de cada proceso en sus respectivas funciones. Logrando así una ejecución limpia, legible y mantenible.

- Llamadas al sistema utilizadas: *fork(), getpid()*.

```
int main(int argc, char *argv[]) {

    int tiempo = leerArgumento(argc,argv);

    /* ESTRUCTURA VERTICAL: ejec --> A --> B */

    pidEjec = getpid();
    pidA = fork();
    comprobarFork(pidA, "Error en fork de A");
    if (pidA > 0) {
        procesoEjec();
    }

    pidA = getpid();
    pidB = fork();
    comprobarFork(pidB, "Error en fork de B");
    if (pidB > 0) {
        procesoA();
    }

    /* ESTRUCTURA HORIZONTAL: B --> {X, Y, Z} */

    pidB = getpid();
    procesoB(tiempo);
}
```

2. Ejercicio 2

2.1. Enunciado del Ejercicio

Realizar un programa llamado **hacha.c** que divida un archivo en varios trozos con el mismo nombre y extensión h00, h01, … . Se debe tener en cuenta el formato y consideraciones siguientes:

$ hacha <archivo> <tamaño>

<archivo> nombre del archivo a dividir

<tamaño> tamaño en bytes de los archivos divididos

Para dividir, el proceso hacha (proceso padre) generará tantos procesos hijos como archivos se tengan que crear. Mediante tuberías, el proceso padre enviará la información a los hijos y serán estos últimos los que creen los archivos de destino y escriban en ellos.

2.1.1. Estructura del Árbol de Procesos



2.1.2. Resultado Esperado

```
$ hacha at_madrid.mp3 50000
$ ls
at_madrid.mp3.h00
at_madrid.mp3.h01
at_madrid.mp3.h02
```

2.2. Identificación y Justificación de Tareas

Para cumplir con las restricciones del enunciado y asegurar un diseño robusto y modular, la resolución se divide en las siguientes tareas.

2.2.1. Validación de Argumentos

El programa implementa la función de validación ***leerArgumentos()*** para garantizar que los argumentos introducidos por terminal sean correctos. Se comprueba que haya exactamente dos argumentos y que el segundo sea mayor que 0.

2.2.2. Cálculo de Fragmentación

Previo a leer el archivo, se debe emplear la llamada ***stat()*** dentro de la función ***TamArchivoOrigen()*** para conocer el tamaño total en bytes del archivo que se va a dividir. Con dicho valor, la función ***calculoNhijos()*** determina cuántos subprocesos (archivos destino) se requieren.

2.2.3. E/S a bajo nivel

El enunciado requiere realizar las entradas/salidas con llamadas al sistema por lo que se emplea ***open()*** y el flag **O_RDONLY**. Por otro lado, los fragmentos se deben realizar mediante ***creat()***, y el intercambio de información, byte a byte, se ejecuta mediante ***read()*** y ***write()***.

2.2.4. Comunicación entre procesos (Tuberías)

Para comunicar el proceso padre con sus respectivos hijos se ha implementado un esquema de tuberías unidireccionales (padre → hijo) mediante ***pipe()***. Para cada fragmento a generar, el padre crea una nueva tubería para transferir la información del segmento requerido.

2.2.5. Ejecución Concurrente y Sincronización final

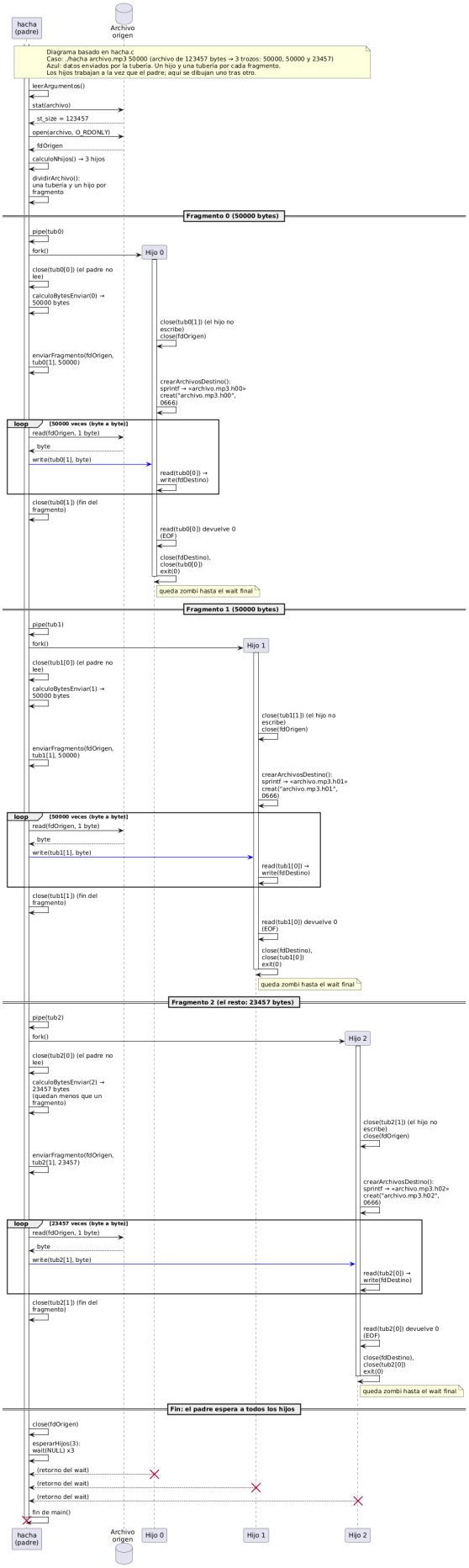
El padre ejecuta en ***dividirArchivo()*** una estructura iterativa donde se crea un hijo, se le pasa la información por la tubería y se pasa a la siguiente iteración para crear al siguiente hijo. La sincronización se delega al final del programa en la función ***esperarHijos()***, asegurando que la muerte de los hijos sea previa a la finalización del super padre.

2.3. Diagrama de Secuencia

El siguiente diagrama muestra el funcionamiento de hacha para el caso *./hacha archivo.mp3 50000*, con un archivo de *123457 bytes* que se divide en 3 fragmentos.

El proceso padre obtiene el tamaño del archivo con **stat**, calcula cuántos hijos necesita y, por cada fragmento, crea una tubería y un hijo con **fork**. El padre cierra el extremo de lectura de la tubería y el hijo el de escritura. A continuación el padre lee el fragmento del archivo origen y lo envía por la tubería (flechas azules), y el hijo lo recibe y lo escribe en su archivo destino (.h00, .h01, ...), creado con **creat**. Cuando el padre cierra su extremo de escritura, el hijo recibe el fin de archivo (read devuelve 0), cierra sus descriptores y termina.

Al acabar de repartir todos los fragmentos, el padre espera a sus hijos con **wait**, de modo que ningún hijo queda huérfano y el padre es el último en terminar.


hacha (padre)
Archivo origen
Diagrama basado en hacha.c
Caso: ./hacha archivo.mp3 50000 (archivo de 123457 bytes → 3 trozos: 50000, 50000 y 23457)
Azul: datos enviados por la tubería. Un hijo y una tubería por cada fragmento.
Los hijos trabajan a la vez que el padre; aquí se dibujan uno tras otro.
leerArgumentos()
stat(archivo)
st_size = 123457
open(archivo, O_RDONLY)
fdOrigen
calculoNhijos() → 3 hijos
dividirArchivo():
una tubería y un hijo por fragmento
Fragmento 0 (50000 bytes)
pipe(tub0)
fork()
Hijo 0
close(tub0[0]) (el padre no lee)
calculoBytesEnviar(0) → 50000 bytes
close(tub0[1]) (el hijo no escribe)
close(fdOrigen)
enviarFragmento(fdOrigen, tub0[1], 50000)
crearArchivosDestino():
sprintf → «archivo.mp3.h00»
creat("archivo.mp3.h00", 0666)
loop
[50000 veces (byte a byte)]
read(fdOrigen, 1 byte)
byte
write(tub0[1], byte)
read(tub0[0]) → write(fdDestino)
close(tub0[1]) (fin del fragmento)
read(tub0[0]) devuelve 0 (EOF)
close(fdDestino), close(tub0[0])
exit(0)
queda zombi hasta el wait final
Fragmento 1 (50000 bytes)
pipe(tub1)
fork()
Hijo 1
close(tub1[0]) (el padre no lee)
calculoBytesEnviar(1) → 50000 bytes
close(tub1[1]) (el hijo no escribe)
close(fdOrigen)
enviarFragmento(fdOrigen, tub1[1], 50000)
crearArchivosDestino():
sprintf → «archivo.mp3.h01»
creat("archivo.mp3.h01", 0666)
loop
[50000 veces (byte a byte)]
read(fdOrigen, 1 byte)
byte
write(tub1[1], byte)
read(tub1[0]) → write(fdDestino)
close(tub1[1]) (fin del fragmento)
read(tub1[0]) devuelve 0 (EOF)
close(fdDestino), close(tub1[0])
exit(0)
queda zombi hasta el wait final
Fragmento 2 (el resto: 23457 bytes)
pipe(tub2)
fork()
Hijo 2
close(tub2[0]) (el padre no lee)
calculoBytesEnviar(2) → 23457 bytes
(quedan menos que un fragmento)
close(tub2[1]) (el hijo no escribe)
close(fdOrigen)
enviarFragmento(fdOrigen, tub2[1], 23457)
crearArchivosDestino():
sprintf → «archivo.mp3.h02»
creat("archivo.mp3.h02", 0666)
loop
[23457 veces (byte a byte)]
read(fdOrigen, 1 byte)
byte
write(tub2[1], byte)
read(tub2[0]) → write(fdDestino)
close(tub2[1]) (fin del fragmento)
read(tub2[0]) devuelve 0 (EOF)
close(fdDestino), close(tub2[0])
exit(0)
queda zombi hasta el wait final
Fin: el padre espera a todos los hijos
close(fdOrigen)
esperarHijos(3):
wait(NULL) x3
(retorno del wait)
(retorno del wait)
(retorno del wait)
fin de main()
hacha (padre)
Archivo origen
Hijo 0
Hijo 1
Hijo 2

2.4. Análisis Detallado del Código Fuente

2.4.1. Funciones Auxiliares y de Control de Errores

- **Funciones *leerArgumentos()*, *calculoNhijos()* y *calculoBytesEnviar()*:** aíslan las operaciones de comprobación y matemáticas del programa. Se encargan de gestionar y validar los argumentos, determinar el número de subprocesos necesarios para hacer la fragmentación y ajustar el tamaño de envío en la última iteración (del sobrante) – respectivamente.

  - Ninguna llamada al sistema utilizada.

```
int calculoNhijos(long tamArch, int tamFrag) {
    int numHijos = tamArch / tamFrag;
    if (tamArch % tamFrag != 0) {
        numHijos++;
    }
    return numHijos;
}
```

  - Ninguna llamada al sistema utilizada.

```
long calculoBytesEnviar(long tamArch, int tamFrag, int i) {
    long bytesRestantes = tamArch - i*tamFrag;
    long bytesEnviar;
    if (bytesRestantes >= tamFrag) {
        bytesEnviar = tamFrag;
    }
    else {
        bytesEnviar = bytesRestantes;
    }
    return bytesEnviar;
}
```

- **Funciones *comprobarFork()*, *comprobarArchivo()*, *comprobarStat()* y *comprobarPipe()*:** tienen la función de filtros de seguridad. Sirven para capturar el valor de retorno de error (negativo) de las llamadas al sistema ***fork()***, ***creat()***, ***stat()*** y ***pipe()***; para interrumpir la ejecución.

  - Llamadas al sistema utilizadas: *perror(), exit()*.

2.4.2. Tratamiento de Archivos

- Función *TamArchivoOrigen()*: emplea la estructura ***stat()*** para leer los metadatos del archivo y obtener su tamaño exacto en bytes (*infoArchivo.st_size*)

- Llamadas al sistema utilizadas: *stat()*.

```
long TamArchivoOrigen(const char *nombre) {
    struct stat infoArchivo;
    comprobarStat(stat(nombre, &infoArchivo));
    return infoArchivo.st_size;
}
```

- Función *abrirOrigen()*: asigna un descriptor de archivo solo lectura al original introducido por el usuario.

- Llamadas al sistema utilizadas: *open()*.

```
int abrirOrigen(const char *nombre) {
    int fdOrigen = open(nombre, O_RDONLY);
    comprobarArchivo(fdOrigen, "Error al abrir el archivo origen");
    return fdOrigen;
}
```

2.4.3. Comunicación (Tuberías)

- Función *enviarFragmento()*: se ejecuta en el proceso padre. Utilizando un búfer, extrae byte a byte del archivo origen y lo inyecta en la tubería. Al acabar, cierra la tubería para enviar un **EOF** implícito al hijo.

- Llamadas al sistema utilizadas: *read(), write(), close()*.

```
void enviarFragmento(int fdOrigen, int tuberia, long bytesEnviar) {
    char buffer[1];

    for (long i=0; i<bytesEnviar; i++) {
        read(fdOrigen,buffer,1);
        write(tuberia,buffer,1);
    }

    close(tuberia);
}
```

- Función *crearArchivosDestino()*: se ejecuta en los procesos hijos. Genera dinámicamente el nombre del archivo con la extensión correspondiente (.h00, .h01...) mediante ***sprintf()***. Abre el archivo destino y, mediante una estructura iterativa, extrae los datos de la tubería y los vuelca en los archivos correspondientes.

- Llamadas al sistema utilizadas: *creat(), read(), write(), close(), exit()*.

```
void crearArchivosDestino(int tuberia, const char *nombre, int i) {
    char buffer[1];

    char nombreDestino[200];
    sprintf(nombreDestino,"%s.h%02d",nombre,i);

    int fdDestino = creat(nombreDestino,0666);
    comprobarArchivo(fdDestino, "Error al crear el archivo destino");

    while (read(tuberia,buffer,1) > 0) {
        write(fdDestino,buffer,1);
    }

    close(fdDestino);
    close(tuberia);
    exit(0);
}
```

2.4.4. Bloque Principal

- Función *dividirArchivo()*: la función más importante del programa. Itera tantas veces como fragmentos se necesiten, en cada una de ellas genera una tubería y un ***fork()***. Utiliza estructuras condicionales para manejar por separado al padre (que calcula el tamaño y envía los datos) y al hijo (crea y escribe en los archivos destino los datos recibidos).

- Llamadas al sistema utilizadas: *pipe(), fork(), close()*.

```
void dividirArchivo(int fdOrigen, const char *nombre, long tamArchivo,
int tamFragmento, int numHijos) {
    for (int i=0; i<numHijos; i++) {
        int tub[2];

        comprobarPipe(pipe(tub));
        pid_t pid = fork();
        comprobarFork(pid, "Error en fork");
        if (pid > 0) {
```

```
            close(tub[0]);
            long bytesEnviar = calculoBytesEnviar(tamArchivo,
tamFragmento, i);
            enviarFragmento(fdOrigen,tub[1],bytesEnviar);
        }
        else {
            close(tub[1]);
            close(fdOrigen);

            crearArchivosDestino(tub[0],nombre,i);
        }
    }
}
```

- **Función *esperarHijos()*:** se encarga de invocar bloqueos sucesivos con ***wait(NULL)*** para garantizar una terminación limpia y sin procesos colgando. Se realiza mediante una estructura iterativa que va desde 0 hasta el número de procesos hijo creados.

    - Llamadas al sistema utilizadas: *wait()*.

2.4.5. Función Principal

- **Función *main()*:** únicamente llama a cada función en el orden adecuado, dichas funciones son las que se encargan del correcto funcionamiento del programa. El main solo sirve como mediador del flujo del programa.

    - Llamadas al sistema utilizadas: close().

```
int main(int argc, char *argv[]) {
    char *nombre;
    int tamFragmento;

    leerArgumentos(argc, argv, &nombre, &tamFragmento);
    long tamArchivo = TamArchivoOrigen(nombre);

    int fdOrigen = abrirOrigen(nombre);
    int numHijos = calculoNhijos(tamArchivo, tamFragmento);

    dividirArchivo(fdOrigen, nombre, tamArchivo, tamFragmento, numHijos);

    close(fdOrigen);
    esperarHijos(numHijos);
}
```

3. Ejercicio 3

3.1. Enunciado del Ejercicio

Diseña un programa llamado hijos.c que cree un árbol de procesos según la siguiente estructura a partir de dos parámetros

$hijos x y

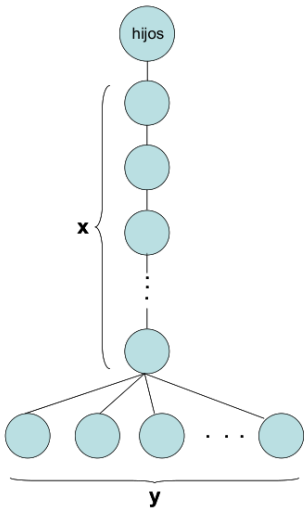
El proceso hijos debe mostrar el siguiente mensaje:

"Soy el superpadre (pid) : mis hijos finales son: pidx1, pidx2, pidx3, ..., pidxy"

Los procesos de la parte baja deben mostrar el siguiente mensaje:

"Soy el subhijo pidx1, mi padres son: pid1, pid2, ..., pidx"

3.1.1. Estructura del Árbol de Procesos



3.1.2. Resultado Esperado

$hijos x y

Soy el subhijo pidx1, mi padres son: pid1, pid2, ..., pidx

Soy el subhijo pidx2, mis padres son: ..., pidx

...

Soy el superpadre (pid) : mis hijos finales son: pidx1, pidx2, pidx3, ..., pidxy"

3.2. Identificación y Justificación de Tareas

Para abordar la complejidad de la memoria compartida y la sincronización, el programa se ha dividido en estas tareas.

3.2.1. Validación de parámetros

Al igual que en los ejercicios anteriores, se debe comprobar que los argumentos son correctos con la función ***leerArgumentos()***.

3.2.2. Creación del Segmento de Memoria

El programa deber reservar un espacio para guardar los PIDs. Esta tarea la realiza la función ***crearMemoria()***, la cual reserva un segmento de memoria para almacenar un array de los 'x' ancestros y las 'y' hojas, utilizando el flag **IPC_PRIVATE**.

3.2.3. Generación Jerárquica y Registro de PIDs

La creación del árbol se realiza desde la función ***creaArbol()*** para los procesos de la columna 'x', la cual a su vez llama a la función ***creaHorizontal()*** para las hojas/procesos de la fila 'y'. Cada proceso debe obtener su propio PID y escribirlo en la posición correspondiente del array de memoria compartida.

3.2.4. Sincronización Reactiva con Alarma

Al estar prohibido el uso de ***sleep()*** para evitar bloquear el programa, la sincronización del final de la creación del árbol con la impresión de los mensajes la hace el último proceso hoja (en la fila 'y') configura un temporizador con ***alarm(1)***. Tras expirar este segundo, su manejador envía una señal **SIGUSR1** al super padre, avisándole de que la memoria compartida está completamente rellenada y lista.

3.2.5. Destrucción Ordenada y Liberación de Recursos

Como en anteriores ejercicios, la muerte de procesos se propaga mediante la técnica de cascada. El padre original se despierta, imprime su mensaje y envía un ***kill()*** a su hijo directo, bloqueándose, tras ello, en ***wait(NULL)***. Esto se repite en todos los procesos hacia abajo en el árbol.

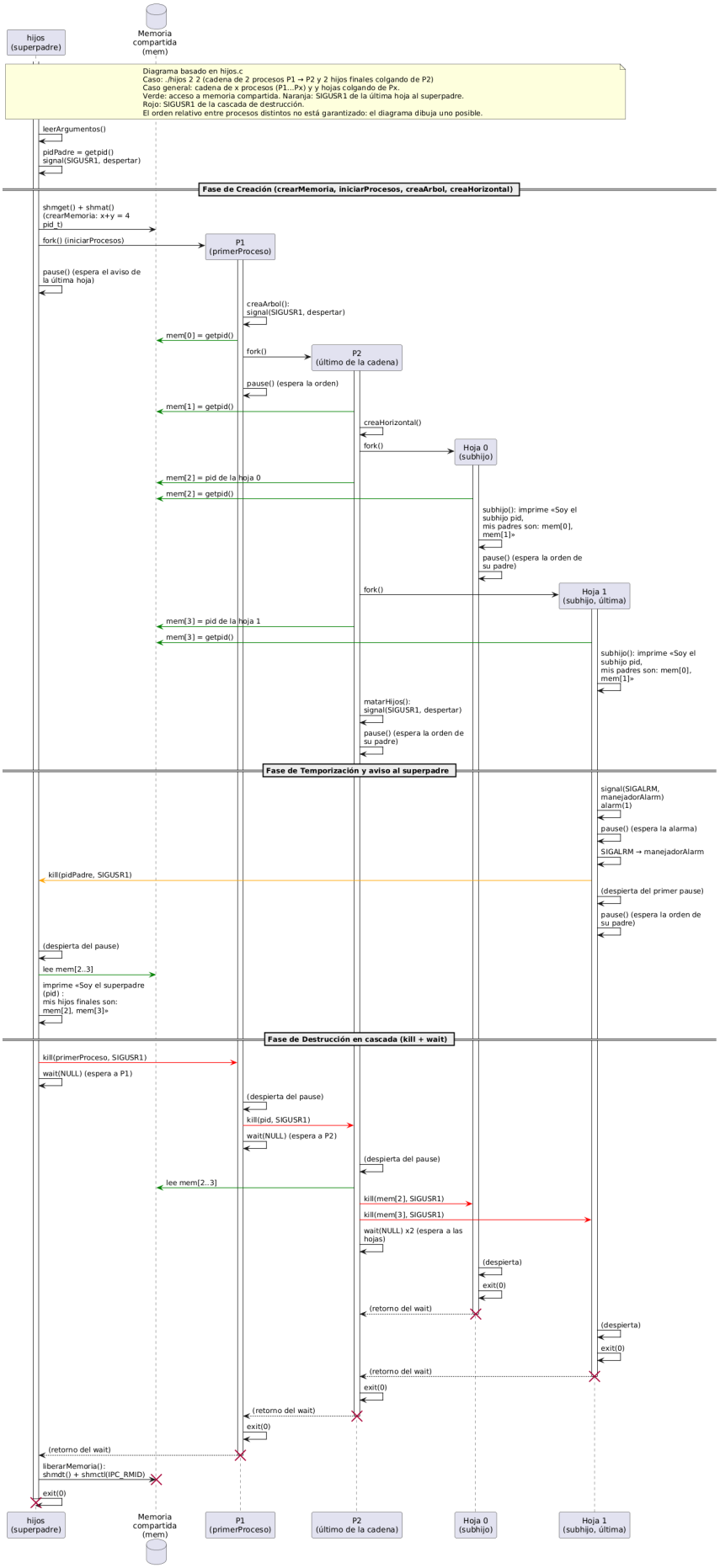
Además, el proceso super padre invoca ***liberarMemoria()*** para desacoplar y destruir el segmento de memoria compartida, evitando fugas indeseadas en el sistema operativo.

3.3. Diagrama de Secuencia

El siguiente diagrama muestra el ciclo de vida completo de hijos para el caso *./hijos 2 2*.

El superpadre crea un segmento de memoria compartida con **shmget** y **shmat**, donde se guardarán los PID, y lanza el primer proceso de la cadena. Cada proceso de la cadena registra su PID en la memoria compartida y crea al siguiente, y el último crea los 'y' hijos finales, que también registran su PID e imprimen el de sus padres leyéndolos de la memoria. El último hijo final programa una **alarm(1)** y, al sonar, su manejador avisa al superpadre con **SIGUSR1** (flecha naranja). El superpadre imprime entonces los PID de los hijos finales, que lee de la memoria compartida, e inicia la destrucción: cada padre pasa **SIGUSR1** a su hijo y espera con **wait** (flechas rojas), y el último proceso de la cadena señala a todas las hojas.

Así mueren primero los hijos finales, después la cadena de abajo arriba, y el superpadre es el último: libera la memoria compartida y termina.


hijos (superpadre)
Memoria compartida (mem)
Diagrama basado en hijos.c
Caso: ./hijos 2 2 (cadena de 2 procesos P1 → P2 y 2 hijos finales colgando de P2)
Caso general: cadena de x procesos (P1...Px) y y hojas colgando de Px.
Verde: acceso a memoria compartida. Naranja: SIGUSR1 de la última hoja al superpadre.
Rojo: SIGUSR1 de la cascada de destrucción.
El orden relativo entre procesos distintos no está garantizado: el diagrama dibuja uno posible.
leerArgumentos()
pidPadre = getpid()
signal(SIGUSR1, despertar)
Fase de Creación (crearMemoria, iniciarProcesos, creaArbol, creaHorizontal)
shmget() + shmat()
(crearMemoria: x+y = 4
pid_t)
fork() (iniciarProcesos)
P1 (primerProceso)
pause() (espera el aviso de la última hoja)
creaArbol():
signal(SIGUSR1, despertar)
mem[0] = getpid()
fork()
P2 (último de la cadena)
pause() (espera la orden)
mem[1] = getpid()
creaHorizontal()
fork()
Hoja 0 (subhijo)
mem[2] = pid de la hoja 0
mem[2] = getpid()
subhijo(): imprime «Soy el subhijo pid, mis padres son: mem[0], mem[1]»
pause() (espera la orden de su padre)
fork()
Hoja 1 (subhijo, última)
mem[3] = pid de la hoja 1
mem[3] = getpid()
subhijo(): imprime «Soy el subhijo pid, mis padres son: mem[0], mem[1]»
matarHijos():
signal(SIGUSR1, despertar)
pause() (espera la orden de su padre)
Fase de Temporización y aviso al superpadre
signal(SIGALRM, manejadorAlarm)
alarm(1)
pause() (espera la alarma)
SIGALRM → manejadorAlarm
kill(pidPadre, SIGUSR1)
(despierta del primer pause)
pause() (espera la orden de su padre)
(despierta del pause)
lee mem[2..3]
imprime «Soy el superpadre (pid) :
mis hijos finales son:
mem[2], mem[3]»
Fase de Destrucción en cascada (kill + wait)
kill(primerProceso, SIGUSR1)
wait(NULL) (espera a P1)
(despierta del pause)
kill(pid, SIGUSR1)
wait(NULL) (espera a P2)
(despierta del pause)
lee mem[2..3]
kill(mem[2], SIGUSR1)
kill(mem[3], SIGUSR1)
wait(NULL) x2 (espera a las hojas)
(despierta)
exit(0)
(retorno del wait)
(despierta)
exit(0)
(retorno del wait)
exit(0)
(retorno del wait)
exit(0)
(retorno del wait)
liberarMemoria():
shmdt() + shmctl(IPC_RMID)
exit(0)
hijos (superpadre)
Memoria compartida (mem)
P1 (primerProceso)
P2 (último de la cadena)
Hoja 0 (subhijo)
Hoja 1 (subhijo, última)

3.4. Análisis Detallado del Código Fuente

3.4.1. Funciones de Control y Utilidad

- **Funciones *comprobarFork()*, *comprobarMemoria()* y *comprobarShmat()*:** actúan como validadores y capturadores de errores del sistema. Si las llamadas fallan, detienen la ejecución y reportan el error a través de ***perror()***.

  - Llamadas al sistema utilizadas: *perror(), exit()*.

- **Función *imprimirLista()*:** función auxiliar encargada de recorrer un segmento del array de memoria y formatear la salida que pide el enunciado, separando los PIDs por comas.

  - Ninguna llamada al sistema utilizada.

```
void imprimirLista(pid_t lista[], int n) {
    for (int i=0; i<n; i++) {
        if (i > 0) {
            printf(",");
        }
        printf(" %d", lista[i]);
    }
    printf("\n");
}
```

- **Función *manejadorAlarm()*:** función que permite interrumpir la espera de procesos activada por el temporizador. Se encarga de enviar el aviso **SIGUSR1** hacia la cima de la jerarquía (al proceso padre).

  - Llamadas al sistema utilizadas: *kill()*.

3.4.2. Gestión de Memoria Compartida

- **Función *crearMemoria()*:** crea un bloque de memoria para guardar los PIDs, que son de tipo "pid_t". El flag **IPC_PRIVATE** asegura que el segmento sea exclusivo para los parientes del árbol, mientras que **IPC_CREAT|0666** aporta los permisos de lectura/escritura.

  - Llamadas al sistema utilizadas: *shmget(), shmat()*.

```
pid_t *crearMemoria(int x, int y, int *shmid) {
    *shmid = shmget(IPC_PRIVATE,sizeof(pid_t)*(x+y), IPC_CREAT|0666);
```

```
    comprobarMemoria(*shmid);
    pid_t *mem = (pid_t *) shmat(*shmid,0,0);
    comprobarShmat(mem);
    return mem;
}
```

- Función *liberarMemoria()*: invocada por el padre al terminar. Primero desacopla el puntero con ***shmdt()*** y luego ejecuta la destrucción de la memoria mediante ***shmctl()*** y la orden **IPC_RMID**

- Llamadas al sistema utilizadas: *shmdt(), shmctl(), exit()*.

```
void liberarMemoria(pid_t *mem, int shmid) {
    shmdt(mem);
    if (shmctl(shmid, IPC_RMID, NULL) == -1) {
        printf("Error al borrar memoria compartida\n");
    }
    exit(0);
}
```

3.4.3. Creación del Árbol y Destrucción

- Funciones *creaArbol()* y *creaHorizontal()*: se encargan de crear la estructura de procesos. Como ya comentado en las tareas, la primera, ***creaArbol()***, itera para formar la columna vertical de ancestros 'x'. El primer proceso de la cadena y sus descendientes guardan sus PIDs en los primeros huecos de la memoria.

- Llamadas al sistema utilizadas: *signal(), fork(), pause(), kill(), wait(), exit(), getpid()*.

```
void creaArbol(int nfilas, int ncolumnas, pid_t *mem) {
    signal(SIGUSR1, despertar);
    mem[0] = getpid();
    for (int i=1; i<nfilas; i++) {
        pid_t pid = fork();
        comprobarFork(pid);
        if (pid > 0) {
            pause();
            kill(pid,SIGUSR1);
            wait(NULL);
            exit(0);
        }
        mem[i] = getpid();
    }
    creaHorizontal(ncolumnas,nfilas,mem);
```

```
    matarHijos(ncolumnas, nfilas, mem);
}
```

Al llegar al final, se invoca la segunda, ***creaHorizontal()***, para desplegar las hojas/procesos en horizontal según la 'y'.

- Llamadas al sistema utilizadas: *fork()*.

```
void creaHorizontal(int ncolumnas, int nfilas, pid_t *mem) {
    for (int i=0; i<ncolumnas; i++) {
        pid_t pid = fork();
        comprobarFork(pid);
        if (pid == 0) {
            int esUltimo = 0;
            if (i == ncolumnas-1) {
                esUltimo=1;
            }
            subhijo(i, esUltimo, nfilas, mem);
        }
        mem[nfilas + i] = pid;
    }
}
```

- <u>Función *subhijo()*</u>: la invocan las hojas. Sirve para acceder a la segunda partición de memoria para que guarden sus PIDs. Acceden a la primera partición para imprimir el listado de los de la vertical (ancestros). El último subhijo dispara la función de alarma, tras ello, todos quedan a la espera con pause().

- Llamadas al sistema utilizadas: *getpid(), signal(), alarm(), pause(), exit()*.

```
void subhijo(int i, int ultimo, int nfilas, pid_t *mem) {
    mem[nfilas + i] = getpid();
    printf("Soy el subhijo %d, mis padres son: ", getpid());
    imprimirLista(mem, nfilas);
    if (ultimo) {
        signal(SIGALRM, manejadorAlarm);
        alarm(1);
        pause();
        pause();
        exit(0);
    }
    pause();
    exit(0);
}
```

- Función *matarHijos()*: se encarga de la destrucción controlada de la última fila de procesos. El último proceso vertical (el que contiene las hojas) envía **SIGUSR1** a todas las hojas y después espera con ***wait(NULL)*** a cada una. Se hace mediante dos estructuras iterativas que acaban con el número de columnas introducido por el usuario.

- Llamadas al sistema utilizadas: *signal(), pause(), kill(), wait(), exit()*.

3.4.4. Función Principal

- Función *main()*: al igual que en el resto, sirve para controlar el flujo y llamar a las funciones correspondientes cuando sea preciso. Empieza con la validación de argumentos para después invocar a ***crearMemoria()***. Tras ello, dispara la creación del árbol mediante ***iniciarProcesos()***. Se mantiene suspendido (el super padre) hasta que la señal de alarma le despierta. Finalmente, envía la orden de muerte a su primer hijo (iniciando la cascada), lo recoge con ***wait(NULL)*** y acaba el programa destruyendo el segmento de memoria con ***liberarMemoria()***.

- Llamadas al sistema utilizadas: *getpid(), pause(), kill(), wait()*.

```
int main(int argc, char *argv[]) {
    int x, y, shmid;
    pid_t *mem, primerProceso;
    leerArgumentos(argc, argv, &x, &y);

    pidPadre = getpid();
    signal(SIGUSR1, despertar);

    mem = crearMemoria(x, y, &shmid);
    primerProceso = iniciarProcesos(x, y, mem);

    pause();

    printf("Soy el superpadre (%d) : mis hijos finales son: ", getpid());
    imprimirLista(&mem[x], y);

    kill(primerProceso, SIGUSR1);
    wait(NULL);
    liberarMemoria(mem, shmid);
}
```

4. Anexo

4.1 Código fuente

Enlace a mi repositorio público en GitHub, que contiene tanto los ejemplos de la práctica como las resoluciones a los ejercicios:

➔ https://github.com/aleandreu/sistemas-operativos-i2ade

4.2 Herramientas Utilizadas

Para la resolución de esta práctica he utilizado:

- **Word**: creación de la memoria

- **VS Code**: compilación y ejecución de los códigos fuente

- **Plant Text**: creación de los diagramas de secuencia en UML

4.3 Uso de herramientas IA

A lo largo de mi resolución de la práctica he utilizado herramientas de IA para 3 cosas en concreto:

1. Explicación de conceptos y ejemplos de la práctica
2. Ejecución de pruebas diversas para verificar el correcto funcionamiento de mis códigos fuente
3. Desarrollo guiado de los diagramas de secuencia en UML